

DOM



CONTENT

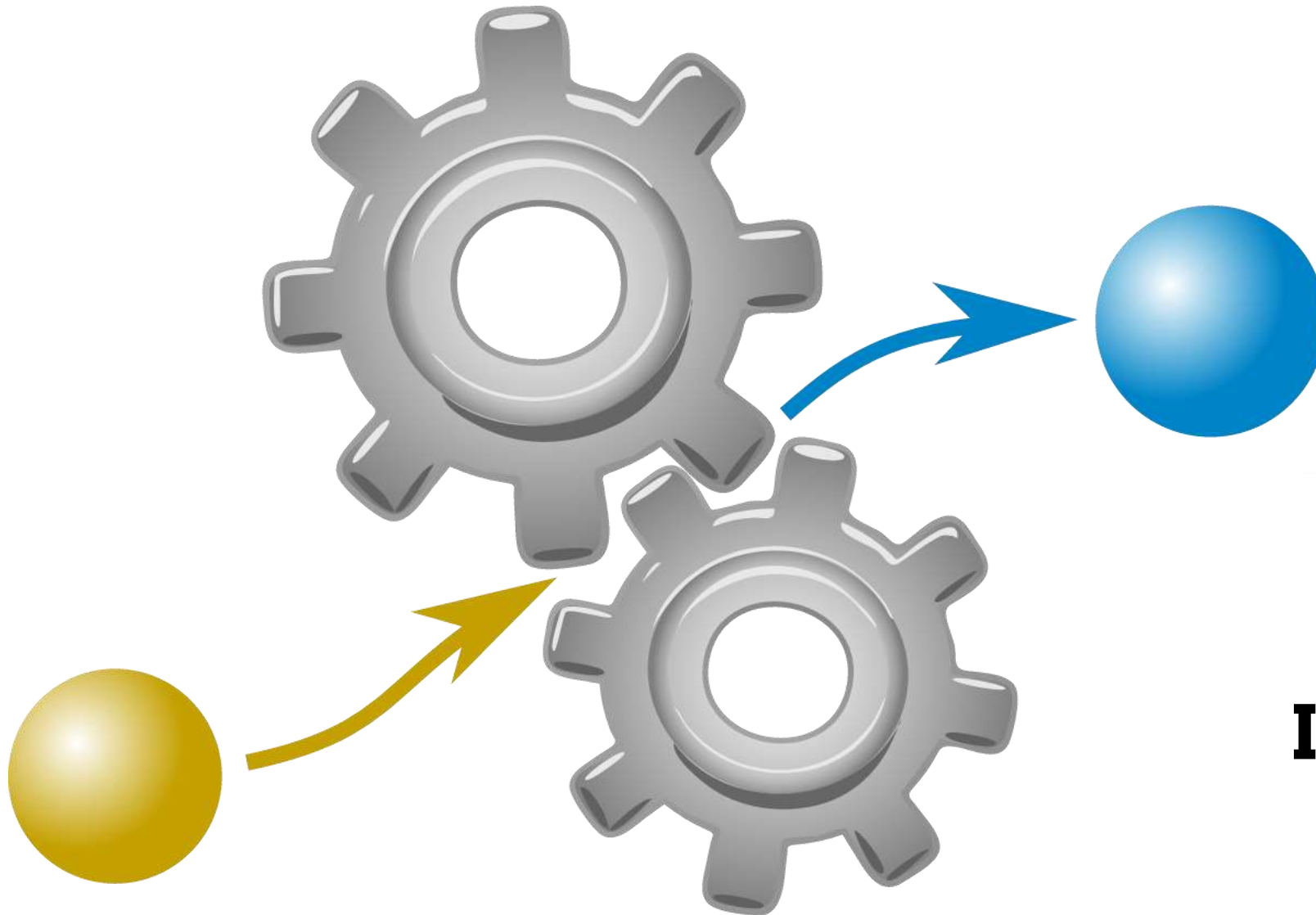
01 Introduction

02 Nodes

03 Selectors

04 Event





01

Introduction

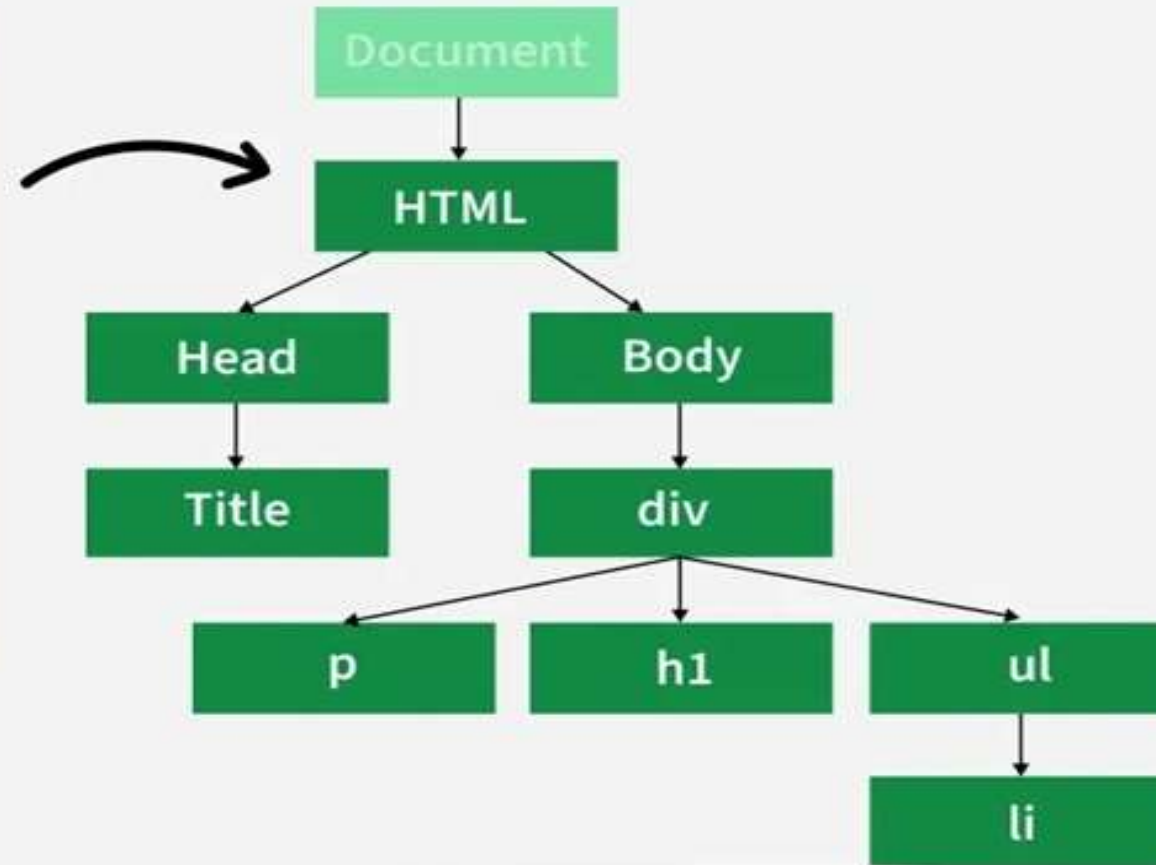
Introduction

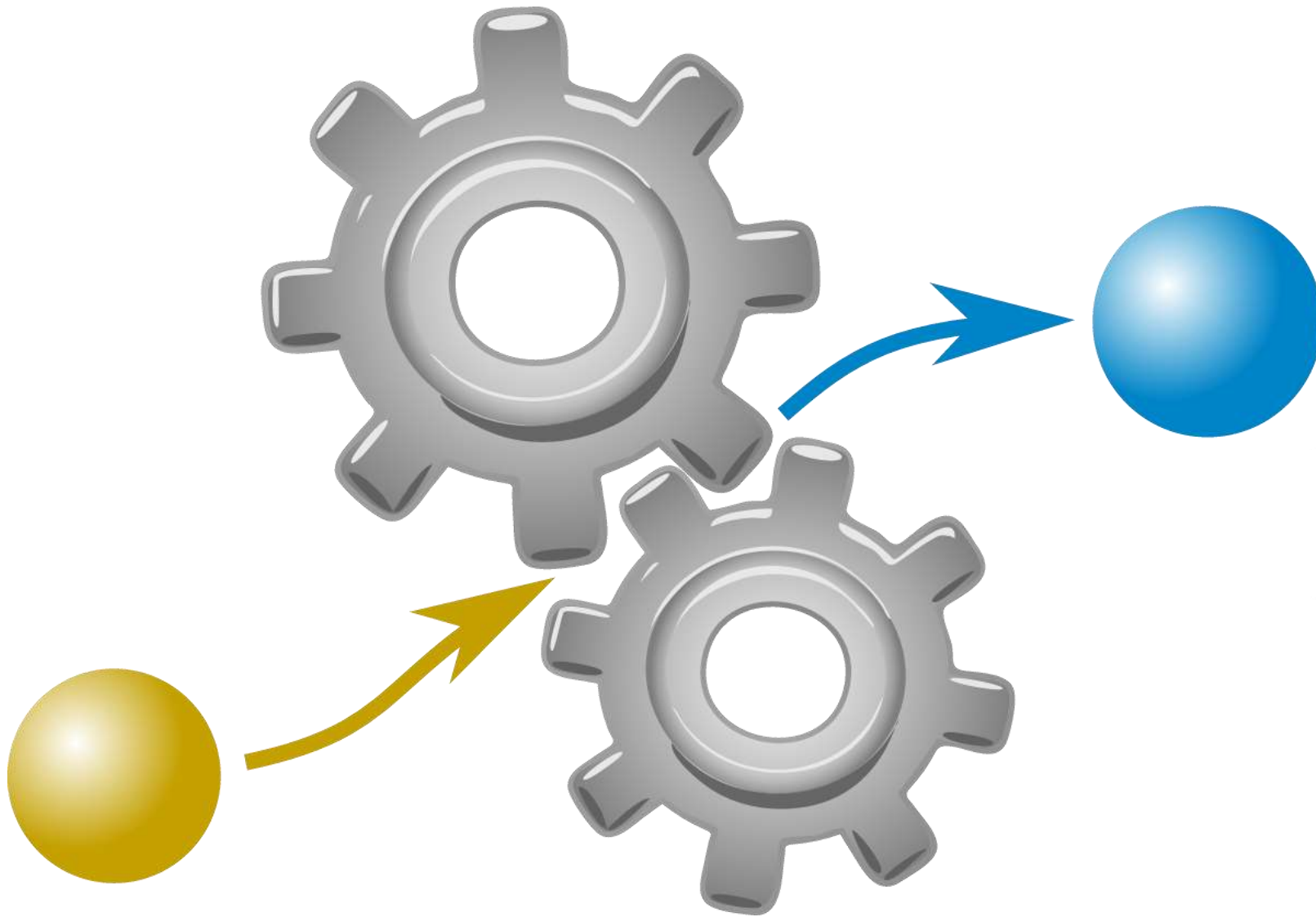


DOM (Document Object Model) is a programming interface provided by the browser that allows JavaScript to interact with and manipulate HTML and CSS.

The “DOM Tree”

```
<html>
  <head>
    <title> Webpage Title</title>
  </head>
  <body>
    <div>
      <h1>This is Heading Tag</h1>
      <p>Some Text Content</p>
      <ul>
        <li> List Item</li>
      </ul>
    </div>
  </body>
</html>
```





02

Nodes

What is Node?

Everything in an HTML document is represented as nodes. These nodes form a tree-like structure called the DOM Tree.

Types of DOM Nodes :

Node Type	Description
Element Node	Represents an HTML element (<div>, <p>)
Attribute Node	Represents an attribute (class="abc")
Text Node	Text inside an element or attribute
Document Node	Represents the entire document (document)



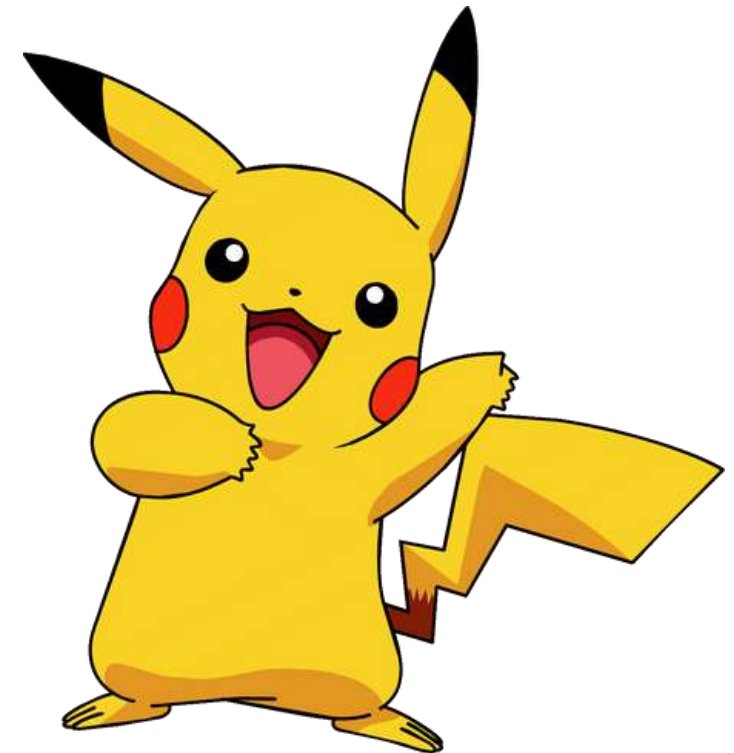
How Browser Uses DOM

Step 1: Browser Loads HTML File

When you open a website, the browser **downloads the HTML file** from the server.

Step 2: HTML Parsing

- The browser reads the HTML **line by line** from top to bottom.
- It starts **building the DOM Tree** — a structured representation of all elements on the page.



Html page :

```
<html>
  <head>
    <title>Page</title>
  </head>
  <body>
    <h1>Hello DOM</h1>
  </body>
</html>
```

This becomes:

CSS

Document

```
└─ html
    └─ head
    └─ body
        └─ h1
```



Step 3: CSS Parsing

- Simultaneously, the browser reads any CSS files (or <style> tags).
- It builds the CSSOM (CSS Object Model) — a tree-like structure that stores all the styles.

Step 4: Combine DOM + CSSOM → Render Tree

- Browser **combines the DOM + CSSOM** into a **Render Tree**.
- Render Tree = Structure + Style

Example

```
<h1 style="color: red">Hello</h1>
```

- DOM says: there is <h1>
CSSOM says: it's red
Render Tree says: display <h1> with red color.



Step 5: Layout & Painting

- Browser **calculates positions** (Layout).
- Then **paints pixels** on the screen (Painting).

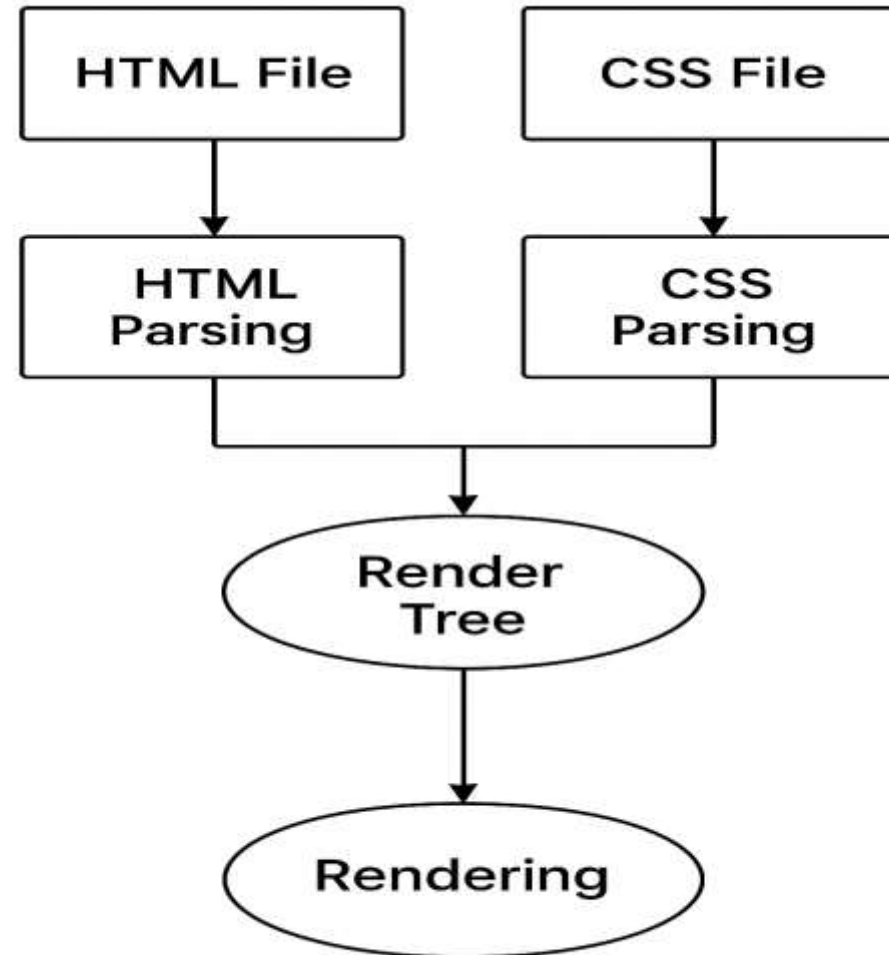
Step 6: JavaScript Can Change DOM

- Using **DOM API**, JavaScript can:
- Change content (innerText, innerHTML)
- Change style (style.color)
- Create/Delete elements (createElement, remove)

Browser then **updates Render Tree** → **Repaint** happens → Page updates **without refreshing**.



Flow Chart :



DOM Selectors

Why Use DOM Selectors ?

- To access and manipulate elements on a webpage using JavaScript

Example: Change text, style, or add/remove elements dynamically.

1. `getElementById()`

Use: Select one element using its ID.

```
html

<p id="demo">Hello</p>
<script>
let element = document.getElementById("demo");
element.innerText = "Changed!";
</script>
```

Return Type: Single Element (Object)



DOM Selectors

2. `getElementsByClassName()`

Use: Select multiple elements using their class name.

html

```
<p class="text">One</p>
<p class="text">Two</p>
<script>
let elements = document.getElementsByClassName("text");
elements[0].innerText = "Updated";
</script>
```

Return Type: **HTMLCollection** (Array-like object)



DOM Selectors

3. `getElementsByTagName()`

Use: Select all elements of a particular HTML tag.

html

```
<p>One</p><p>Two</p>  
<script>  
let tags = document.getElementsByTagName("p");  
tags[1].innerText = "Second Paragraph";  
</script>
```

Return Type: `HTMLCollection`



DOM Selectors

4. `querySelector()`

Use: Select the first matching element using CSS selector.

```
html

<p class="text">One</p>
<p class="text">Two</p>
<script>
let el = document.querySelector(".text");
el.style.color = "red";
</script>
```

Return Type: Single Element (First Match)



DOM Selectors

5. `querySelectorAll()`

Use: Select all matching elements using CSS selector.

```
html

<p class="text">One</p>
<p class="text">Two</p>
<script>
let all = document.querySelectorAll(".text");
all.forEach((item) => item.style.color = "blue");
</script>
```

Return Type: `NodeList` (supports `forEach()`)



Comparison Table



Selector	Single/Multiple	Return Type	CSS Selector Support
getElementById	Single	Element	✗ No
getElementsByClassName	Multiple	HTMLCollection	✗ No
getElementsByTagName	Multiple	HTMLCollection	✗ No
querySelector	Single	Element	✓ Yes
querySelectorAll	Multiple	NodeList	✓ Yes



HTML Collection

Html :

```
<div class="fruit">Apple</div>
<div class="fruit">Banana</div>
<div class="fruit">Mango</div>
<button onclick="play()">Click Me</button>
```

js

Copy Edit

```
function play() {
  let collection = document.getElementsByClassName('fruit'); // HTMLCollection
  console.log(collection); // HTMLCollection looks like an array but is not a real array.

  console.log("First Item:", collection[0].innerText); // Access using index

  // ❌ collection.forEach() doesn't work directly
  // ✅ Convert to Array
  let array = Array.from(collection);
  array.forEach(item => item.style.color = "red");
}
```



NodeList

js

 Copy  Edit

```
function play() {  
  let nodeList = document.querySelectorAll('.fruit'); // NodeList  
  console.log(nodeList); // NodeList  
  
  //  Access using index  
  console.log("Second Item:", nodeList[1].innerText);  
  
  //  forEach works directly  
  nodeList.forEach(item => item.style.fontWeight = "bold");  
}
```



DOM Manipulation – CRUD Concept



CRUD stands for:

- **C** → Create (Add elements)
- **R** → Read (Access elements)
- **U** → Update (Change elements)
- **D** → Delete (Remove elements)

In DOM, you can use JavaScript to perform all these operations dynamically.



Create – Adding New Elements

Method

- `document.createElement()`
- `element.appendChild()`
- `element.append()`

Example :

html

 Copy  Edit

```
<div id="container"></div>
<script>
let newPara = document.createElement("p");
newPara.innerText = "This is a new paragraph!";
document.getElementById("container").appendChild(newPara);
</script>
```



Read – Access Existing Elements

Selectors

- `getElementById()`
- `querySelector()`

Example :

html

 Copy  Edit

```
<p id="readMe">Hello Students!</p>
<script>
let element = document.getElementById("readMe");
console.log("Text:", element.innerText);
</script>
```



Update – Change Content/Style/Attributes

Methods

- `innerText`, `innerHTML` to update content
- `style.property` to change CSS
- `setAttribute()` to change attributes

Example :

html

 Copy  Edit

```
<p id="updateMe">Old Text</p>
<script>
let element = document.getElementById("updateMe");
element.innerText = "Updated Text!";
element.style.color = "green";
</script>
```



Delete – Remove Elements

Methods

- `element.remove()`
- `parentElement.removeChild()`

Example :

html

 Copy  Edit

```
<p id="deleteMe">Goodbye!</p>
<button onclick="deletePara()">Delete</button>
<script>
function deletePara() {
    document.getElementById("deleteMe").remove();
}
</script>
```





DOM Events

A DOM Event is an action or occurrence in the browser you can respond to using JavaScript.

Examples of events:

- **click**
- **mouseover**
- **keydown**
- **submit**

Note :

Think of events as signals that something has happened on your webpage (e.g., a button was clicked).



Common Types of Events

Event Type	Description	Example
click	Mouse click on element	Button click
dblclick	Double click	Double clicking text
mouseover	Mouse hovers over element	Hover effect on image
mouseout	Mouse leaves element	Remove hover effect
keydown	Keyboard key pressed	Typing in form field
keyup	Key released	Detect key release
submit	Form submission	Submit button on form
load	When page fully loads	Loading animation finished



How to Use DOM Events

1. Using HTML Attribute (Inline Event)

html

 Copy  Edit

```
<button onclick="alert('Clicked!')">Click Me</button>
```

2. Using JavaScript addEventListener() (Recommended Way)

html

 Copy  Edit

```
<button id="btn">Click</button>
```

```
<script>
```

```
document.getElementById("btn").addEventListener("click", function() {  
    alert("You clicked the button!");
```

```
});
```

```
</script>
```

Why better? → Separates **HTML** and **JavaScript**, promotes **clean code**.



Event Listener

Syntax :

```
element.addEventListener(event, function);
```



Example :

js

 Copy  Edit

```
document.querySelector("h1").addEventListener("mouseover", () => {  
    console.log("Mouse is over heading!");  
});
```



What is the Event Object ?

- The Event Object (commonly named e, event, or anything else) is automatically passed to your event handler function.
- It contains all information about the event that happened — like what was clicked, which key was pressed, or where the mouse was.

Example :

html

Copy Edit

```
<button id="clickMe">Click</button>

<script>
document.getElementById("clickMe").addEventListener("click", function(e) {
    console.log(e); // logs entire event object
});
</script>
```

When you click the button, the browser gives you a detailed event object.



Commonly Used Properties of Event Object

Property	Purpose	Example Usage
e.type	Type of event (click, keyup)	console.log(e.type) → "click"
e.target	Which element triggered event	console.log(e.target)
e.currentTarget	The element the event is attached to	Often used in delegation
e.preventDefault()	Prevents default browser behavior	Prevent form submission
e.stopPropagation()	Stops event from bubbling up	Block event flow
e.clientX, e.clientY	Mouse click X/Y coordinates	Mouse position on screen
e.key	For keyboard events, which key was pressed	console.log(e.key)



Example: Prevent Form Submission

html

 Copy  Edit

```
<form id="myForm">
  <input type="text">
  <button>Submit</button>
</form>

<script>
document.getElementById("myForm").addEventListener("submit", function(e){
  e.preventDefault(); // stops page reload
  alert("Form submission prevented!");
});
</script>
```



Example: Detect Keyboard Press



html

 Copy  Edit

```
<input id="nameField" type="text" placeholder="Type something">

<script>
document.getElementById("nameField").addEventListener("keydown", function(e){
    console.log("You pressed:", e.key);
});
</script>
```



DOM Traversal

DOM Traversal means navigating between parent, child, and sibling elements in the DOM tree.

Think of it like **moving through a family tree**:

- Parent → parent element
- Children → direct nested elements
- Siblings → elements on the same level



HTML Example to Understand DOM Tree

html

Copy Edit

```
<div id="parent">
  <p id="child1">First Paragraph</p>
  <p id="child2">Second Paragraph</p>
</div>
```

1. Parent Traversal

Access the parent element

js

Copy Edit

```
let child = document.getElementById("child1");
console.log(child.parentNode); // Logs <div id="parent">
```

parentNode

Immediate parent of element



HTML Example to Understand DOM Tree

2. Child Traversal

Access child elements.

```
14
15 let parent = document.getElementById("parent");
16 console.log(parent.childNodes);    // Returns NodeList (includes text nodes like spaces)
17 console.log(parent.children);     // Returns only HTML elements
18 console.log(parent.firstChild);   // First element (ignoring text nodes)
19 console.log(parent.lastElementChild); // Last element
20
```

Method	Description
childNodes	All child nodes (text + elements)
children	Only element children
firstChild / lastChild	First/last node (includes text)
firstElementChild	First element only



HTML Example to Understand DOM Tree

3. Sibling Traversal

Move between elements on the same level.

js

 Copy  Edit

```
let first = document.getElementById("child1");  
console.log(first.nextElementSibling); // Second Paragraph  
console.log(first.previousElementSibling); // null (no sibling before first)
```

Method	Meaning
<code>nextElementSibling</code>	Next sibling element
<code>previousElementSibling</code>	Previous sibling element



HTML Example to Understand DOM Tree

4. Access Node Content

[Copy](#) [Edit](#)

```
let para = document.getElementById("child1");
console.log(para.innerText); // Output text
console.log(para.innerHTML); // Output HTML inside paragraph
```

Summary

Action	Method	Example
Parent	parentNode	element.parentNode
Children	children, firstElementChild	element.children[0]
Siblings	nextElementSibling, previousElementSibling	element.nextElementSibling
Content	innerText, innerHTML	element.innerText



Dom Event Flow

What Happens in DOM Event Flow?

When an event (like click) happens, it travels through the DOM in 3 phases:

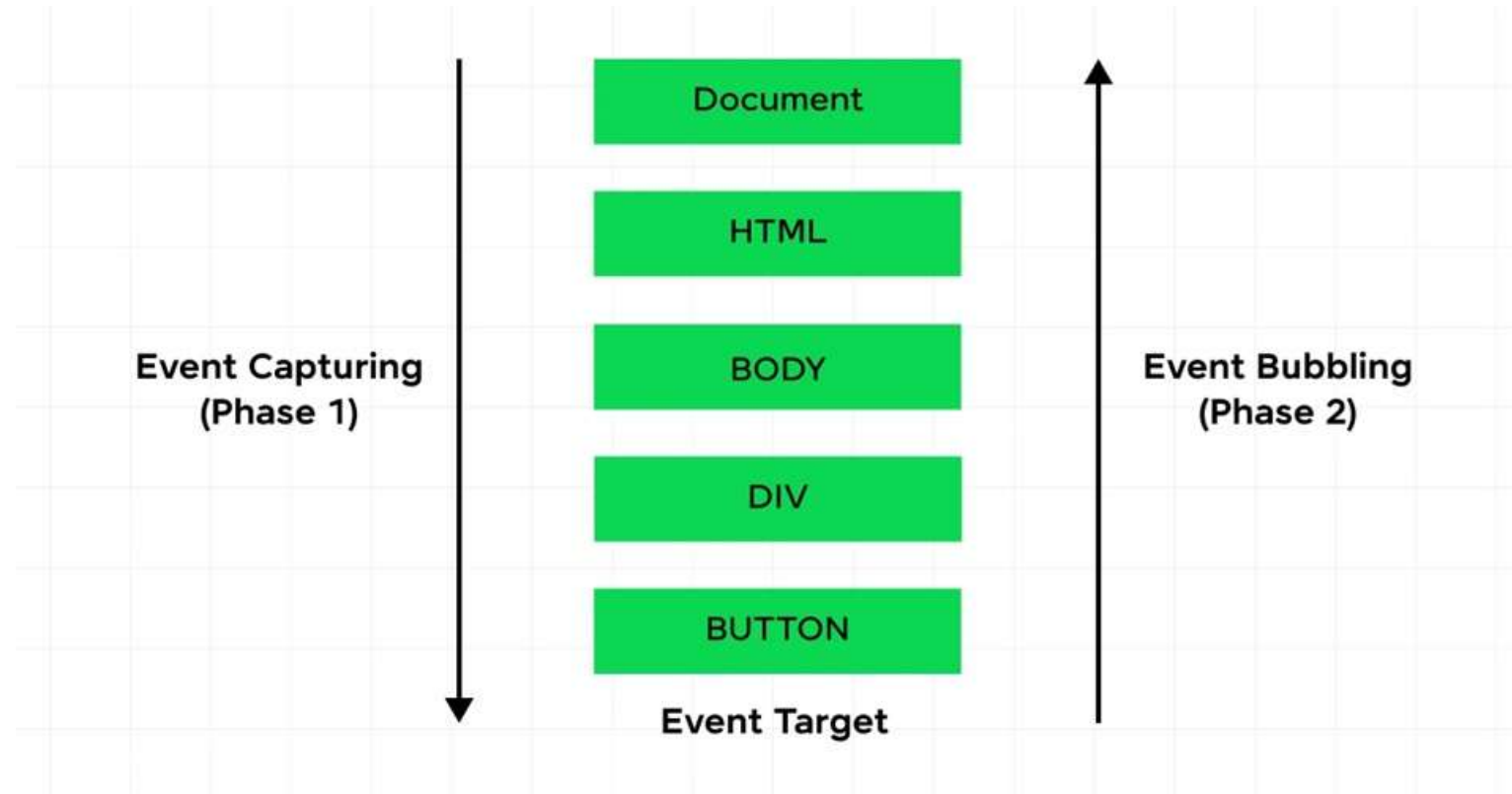
1. **Capture Phase (Capturing)** — Top to Target
2. **Target Phase** — Exact element clicked
3. **Bubble Phase (Bubbling)** — Target to Top

Note :

You can **handle events** in either the **bubbling phase (default)** or the **capturing phase**.



Dom Event Flow



Event Bubbling

```

9      <div id="grand">
10     |   <div id="parent">
11     |   <button id="child">Click Me</button>
12     | </div>
13     | </div>
14

```

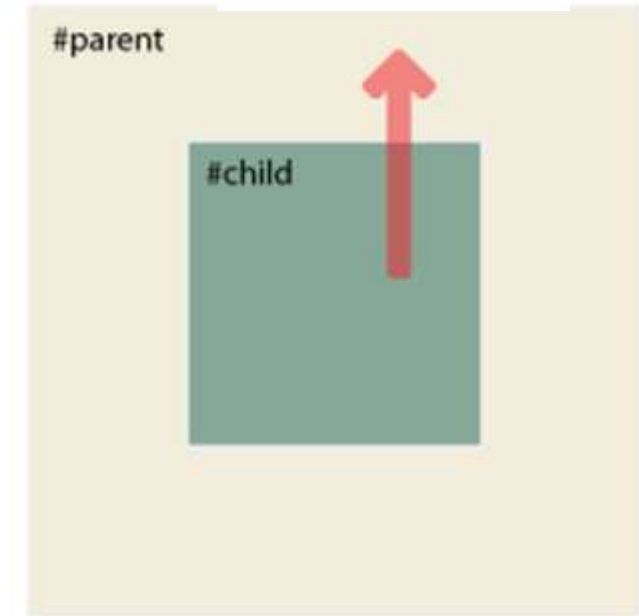
```

<script>
document.getElementById('grand').addEventListener('click',function(){
|   alert('Grand parent clicked!')
})

document.getElementById("parent").addEventListener("click", function(){
|   alert("Parent Clicked");
});

document.getElementById("child").addEventListener("click", function(){
|   alert("Child Clicked");
});
</script>

```



(a) Bubbling Phase

Event Capturing



```

9      <div id="grand">
10     |    <div id="parent">
11     |    <button id="child">Click Me</button>
12     </div>
13     </div>
14

```

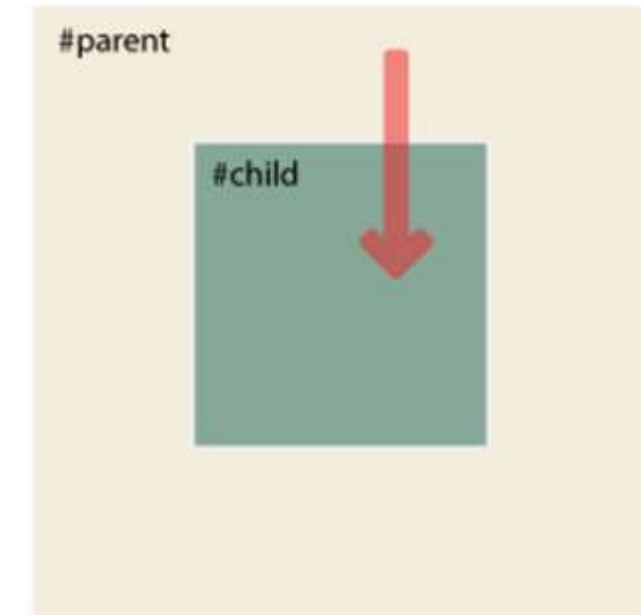
```

<script>
document.getElementById('grand').addEventListener('click',function(){
|   alert('Grand parent clicked!')
|},true)

document.getElementById("parent").addEventListener("click", function(){
|   alert("Parent Clicked");
|},true);

document.getElementById("child").addEventListener("click", function(){
|   alert("Child Clicked");
|},true);
</script>

```



(b) Capturing Phase





Event Delegation

- Instead of adding event listeners to multiple child elements, you add one listener to the parent, and catch events using `e.target`.

Why Use Event Delegation?

- ☐ Improves performance when you have many dynamic elements (e.g., Todo List).
- ☐ Useful when elements are **added dynamically** after page load.



pooh





Event Delegation

```
<ul id="list">
  <li>Apple</li>
  <li>Mango</li>
  <li>Banana</li>
</ul>
```

```
<script>
document.getElementById("list").addEventListener("click", function(e){
  if(e.target.tagName === "LI"){
    alert("You clicked on " + e.target.innerText);
  }
});
</script>
```

Here, we **did NOT** add listeners to every `<li>`, just on `<ul>`



When to Use Event Delegation:



Use Case	Should You Use Delegation?
Static elements (small pages)	✗ Not necessary
Dynamic elements (like comments, tasks)	✓ YES, highly recommended
Performance optimization	✓ Yes

